

UTN • FACULTAD REGIONAL MAR DEL PLATA

Programación II

Desarrollo en Java

Clase N° 2

Métodos, String y Arrays

De la función de C al método de Java, y los dos tipos que más vas a usar

Prof. Daniel Díaz

Ciclo lectivo 2026

```
public class Saludo {  
  
    static String saludar(String n) {  
        return "Hola, " + n;  
    }  
  
    public static void main(String[] a) {  
        System.out.println(saludar("UTN"));  
    }  
}
```

Una función de C, escrita en Java.

Contenidos de la clase

1

De la función de C al método

Cómo se traduce lo que ya sabés hacer

2

Valores y direcciones

Por qué String se comporta como un puntero

3

El tipo String

Cadenas de caracteres sin punteros ni malloc

4

Métodos de String

Comparar, medir, buscar, extraer y transformar

5

Cadenas modificables

Por qué un String no se puede cambiar, y qué usar

6

Arrays

Declarar, reservar, inicializar y recorrer

Toda la clase se apoya en lo que ya sabés de C. Cada tema arranca mostrando cómo lo resolvías antes y cómo se escribe ahora.



De la función de C al método

Lo que en C llamabas función, en Java se llama método

Función en C

Método en Java

Parámetros

void y return

Repaso: la función en Lenguaje C

Repaso de C

Una función agrupa un bloque de instrucciones bajo un nombre, recibe datos y devuelve un resultado.

```
#include <stdio.h>

/* Prototipo: declara la función antes de usarla */
int sumar(int a, int b);

int main() {
    int total = sumar(4, 7);
    printf("Total: %d\n", total);
    return 0;
}

/* Definición: el cuerpo de la función */
int sumar(int a, int b) {
    return a + b;
}
```

Tipo de retorno

int — qué devuelve la función al terminar

Nombre

sumar — con qué se la invoca

Parámetros

(int a, int b) — los datos que recibe

Cuerpo

El bloque entre llaves que se ejecuta

Prototipo

En C hay que declararla antes. En Java NO existe el prototipo.

En Java, el código va dentro de un bloque class

En C escribías las funciones sueltas en el archivo. En Java todo el archivo está envuelto en un bloque que se abre con la palabra class y el nombre del archivo.

Lenguaje C • archivo Calculadora.c

```
int sumar(int a, int b) {  
    return a + b;  
}  
  
int main() {  
    printf("%d", sumar(4, 7));  
    return 0;  
}
```

Java • archivo Calculadora.java

```
public class Calculadora {  
  
    static int sumar(int a, int b) {  
        return a + b;  
    }  
  
    public static void main(String[] args) {  
        System.out.println(sumar(4, 7));  
    }  
}
```

El nombre debe coincidir

El archivo Calculadora.java tiene adentro class Calculadora. Java distingue mayúsculas.

Es el envoltorio del archivo

Por ahora tomalo como el equivalente a tu archivo .c: abre al principio y cierra al final.

Una función se llama método

Es exactamente el mismo concepto. Cambia el nombre porque está adentro del bloque class.

Función de C y método de Java, lado a lado

Aspecto	Función en C	Método en Java
Dónde se escribe	Suelta en el archivo .c	Dentro del bloque class del archivo .java
Declaración previa	Necesita prototipo si se define después del main	No existe el prototipo: el orden no importa
Tipo de retorno	Obligatorio: un tipo o void	Obligatorio: un tipo o void
Paso de parámetros	Por valor; por dirección usando punteros	Siempre por valor: no existen los punteros
Punto de entrada	<code>int main()</code>	<code>public static void main(String[] args)</code>
Cómo se invoca	<code>sumar(4, 7);</code>	<code>sumar(4, 7);</code>
Bibliotecas	<code>#include <stdio.h></code>	<code>import java.util.Scanner;</code>

El concepto es idéntico. Lo único que cambia es dónde se escribe, cómo se llama y un par de palabras extra en la primera línea.

Anatomía de un método

```
static double calcularPromedio(double a, double b) { ... }
```

static

La palabra que permite invocarlo directamente, igual que llamabas a una función en C. La próxima diapositiva la explica.

Tipo de retorno

El tipo del valor que devuelve. Si no devuelve nada, se escribe void. Igual que en C.

Nombre

En lowerCamelCase y empezando con un verbo: calcularPromedio, buscarAlumno, esValido.

Parámetros

Tipo y nombre de cada dato que recibe, separados por coma. Puede no recibir ninguno: ().

Diferencia con C: no hay prototipo

Podés invocar un método que está escrito más abajo en el archivo. El compilador de Java lee todo el bloque class antes de compilar, así que el orden en que escribís los métodos no importa.

La palabra static: por ahora, siempre

Todos los métodos que escribas en esta materia, hasta nuevo aviso, llevan la palabra static.

Qué significa en la práctica

Que el método se puede invocar directamente, escribiendo su nombre. Es el comportamiento que ya conocés de las funciones de C.

Por qué existe la palabra

Java permite otra forma de escribir métodos, que requiere un paso previo antes de invocarlos. Ese tema lo vemos más adelante en la materia.

main también lo lleva

Fijate que la línea de arranque dice public static void main. Es la misma palabra, por la misma razón.

Si te lo olvidás

El compilador tira este error, que es largo y asusta:

```
non-static method sumar(int,int)
cannot be referenced from a
static context
```

La solución

Agregar static a ese método. Cuando veas ese mensaje, casi siempre es eso.

```
int sumar(int a, int b)           // error
static int sumar(int a, int b)    // correcto
```


Un programa completo dividido en métodos

```
public class GestorNotas {  
  
    // Devuelve el promedio de dos notas  
    static double calcularPromedio(double n1, double n2) {  
        return (n1 + n2) / 2;  
    }  
  
    // Devuelve true si el promedio alcanza para aprobar  
    static boolean estaAprobado(double promedio) {  
        return promedio >= 6;  
    }  
  
    // No devuelve nada: solo muestra  
    static void mostrarResultado(String alumno, double prom) {  
        String estado = estaAprobado(prom) ? "APROBADO" : "DESAPROBADO";  
        System.out.println(alumno + " - " + prom + " - " + estado);  
    }  
  
    public static void main(String[] args) {  
        double promedio = calcularPromedio(7.5, 4.5);  
        mostrarResultado("Ana Pérez", promedio);  
    }  
}
```

Cada método hace una sola cosa

Uno calcula, otro decide, otro muestra.

Un método puede llamar a otro

mostrarResultado invoca a estaAprobado.

void no devuelve nada

mostrarResultado imprime, no retorna.

El orden no importa

main está último y usa métodos escritos arriba. En C hubieran hecho falta prototipos.

Salida del programa

Ana Pérez - 6.0 - APROBADO

Parámetros: en Java no hay punteros

El método recibe una copia del valor. Modificar el parámetro adentro del método no cambia la variable original de quien lo llamó.

Lo que NO funciona

```
static void duplicar(int numero) {  
    numero = numero * 2;    // solo cambia la copia  
}  
  
public static void main(String[] args) {  
    int valor = 5;  
    duplicar(valor);  
    System.out.println(valor);    // imprime 5, no 10  
}
```

Lo que sí funciona

```
static int duplicar(int numero) {  
    return numero * 2;    // devuelve el resultado  
}  
  
public static void main(String[] args) {  
    int valor = 5;  
    valor = duplicar(valor);  
    System.out.println(valor);    // imprime 10  
}
```

En C lo resolvías con un puntero

Escribías `void duplicar(int *n)` y llamabas `duplicar(&valor)`. En Java no existen ni el operador `*` ni el `&`, así que la única forma de que un método te devuelva un resultado es con `return`. La excepción son los arrays, y en la sección 6 vemos por qué.

void, return y el valor de salida

void

No devuelve nada

```
static void saludar(String nombre) {  
    System.out.println("Hola, " + nombre);  
}  
  
// Se invoca como una instrucción suelta  
saludar("Ana");
```

return corta la ejecución

Todo lo que esté después de un return que se ejecuta, nunca corre. Igual que en C.

El compilador exige que siempre haya retorno

Si el método declara un tipo, TODOS los caminos posibles deben terminar en return. En C esto solo era una advertencia.

return

Devuelve un valor

```
static String saludo(String nombre) {  
    return "Hola, " + nombre;  
}  
  
// Se invoca donde va un valor  
String s = saludo("Ana");
```

Puede haber varios return

Es habitual devolver temprano cuando un caso ya está resuelto.

En void, return; es válido

Sin valor, sirve para salir del método antes de tiempo.

Cómo escribir métodos que se entiendan



Un método, una responsabilidad

Si al describirlo tenés que usar «y», probablemente sean dos métodos.



El nombre empieza con un verbo

calcularTotal(), buscarPorLegajo(), esMayorDeEdad().



Corto: hasta 20 líneas de cuerpo

Si no entra en la pantalla, cuesta seguirlo.



Pocos parámetros: hasta 3 o 4

Más que eso suele indicar que conviene repensar el método.

Antes

```
static void p(double a, double b) {  
    double x = (a + b) / 2;  
    if (x >= 6) System.out.println("OK");  
    else System.out.println("NO");  
}
```

Después

```
static double calcularPromedio(double n1, double n2) {  
    return (n1 + n2) / 2;  
}  
  
static boolean estaAprobado(double promedio) {  
    return promedio >= 6;  
}  
  
static void mostrarEstado(double promedio) {  
    System.out.println(estaAprobado(promedio) ? "OK" :  
        "NO");  
}
```

2

Valores y direcciones

Por qué String y los arrays se comportan como punteros de C

Tipos de valor

Tipos de referencia

La notación punto

Dos familias de tipos de dato

Los tipos primitivos que viste la clase pasada guardan el valor directamente. String y los arrays guardan otra cosa: una dirección de memoria. Es el mismo mecanismo que ya conocés de los punteros de C.

TIPOS PRIMITIVOS

`int double char boolean long float byte short`

edad

20

La caja contiene el valor.

TIPOS DE REFERENCIA

`String int[] double[] String[]`

ciudad

0x7A3F



"Mar del Plata"

La caja contiene la dirección donde está el dato.

Se escriben con mayúscula inicial

`int` y `double` van en minúscula; `String` siempre con `S` mayúscula. Esa es la pista visual.

No hay `*` ni `&`

Java sigue la dirección solo. Nunca escribís `*ciudad` ni `&ciudad`.

No hay `malloc` ni `free`

La memoria se reserva con `new` y se libera sola cuando nadie la usa más.

`==` compara direcciones

Igual que en C: comparar dos punteros no compara lo que apuntan.

La notación punto: el dato va adelante

En C, las funciones de biblioteca reciben el dato como primer parámetro. En Java el dato se escribe primero, después un punto, y después el nombre de la función.

En C

strlen(ciudad)

La función primero, el dato entre paréntesis.

En Java

ciudad.length()

El dato primero, un punto, y después la función.

Se lee: «de ciudad, dame su longitud». Es la misma operación, escrita al revés.

```
String ciudad = "Mar del Plata";

int largo = ciudad.length();           // 13
String grande = ciudad.toUpperCase();   // "MAR DEL PLATA"
char primera = ciudad.charAt(0);        // 'M'

// Se pueden encadenar: cada punto opera sobre el resultado anterior
String r = ciudad.trim().toUpperCase();
```

Los paréntesis son obligatorios

Aunque la función no reciba nada, hay que escribir (). Sin paréntesis, Java entiende que estás pidiendo un dato guardado, no una función.

Esta distinción vuelve a aparecer en la sección de Arrays, y ahí sí cambia el resultado.

De string.h a los métodos de String

Casi todo lo que hacías con string.h tiene un equivalente directo. Esta tabla es el mapa completo de la sección que sigue.

En C · string.h	En Java	Diferencia a tener en cuenta
<code>strlen(s)</code>	<code>s.length()</code>	Ninguna: devuelve la cantidad de caracteres
<code>strcpy(destino, s)</code>	<code>destino = s;</code>	No hace falta copiar: alcanza con asignar
<code>strcat(destino, s)</code>	<code>destino = destino + s;</code>	El operador + concatena cadenas
<code>strcmp(a, b)</code>	<code>a.compareTo(b)</code>	Mismo criterio de signo: negativo, cero o positivo
<code>strcmp(a, b) == 0</code>	<code>a.equals(b)</code>	En Java es la forma correcta de preguntar «son iguales»
<code>strstr(s, sub)</code>	<code>s.indexOf(sub)</code>	C devuelve un puntero; Java devuelve la posición, o -1
<code>s[i]</code>	<code>s.charAt(i)</code>	Se puede LEER, pero no asignar: <code>s.charAt(0) = 'X'</code> no existe
<code>strtok(s, ";")</code>	<code>s.split(";")</code>	Java devuelve todos los pedazos de una vez, en un arreglo
<code>toupper()</code> en un bucle	<code>s.toUpperCase()</code>	Una sola llamada convierte la cadena entera

Fijate el patrón: donde C pasaba la cadena como parámetro, Java la pone adelante con un punto.

3

El tipo String

Cadenas de caracteres, sin punteros y sin reservar memoria a mano

Qué es

Cómo se declara

Por qué == no sirve

El tipo String

Representa y manipula cadenas de caracteres: letras, números y símbolos. Reemplaza al `char[]` de C, y viene con todas las operaciones ya resueltas.

Se escribe con S mayúscula

Es un tipo de referencia, no un primitivo. La mayúscula es la señal.

No hay que reservar memoria

Nada de `char cadena[100]` ni `malloc`: la cadena crece según lo que le asignes.

No lleva el '\0' al final

Java guarda la longitud aparte. Por eso `length()` es instantáneo y no recorre la cadena.

No se puede modificar carácter a carácter

En C hacías `cadena[0] = 'X'`. En Java eso no existe. Lo vemos en la sección 5.

```
// En C
char ciudad[20] = "Mar del Plata";
int largo = strlen(ciudad);
```

```
// En Java
String ciudad = "Mar del Plata";
int largo = ciudad.length();
```

Cómo se declara un String

```
String materia = "Programación II"; // la forma normal: literal entre comillas
String vacia   = "";                // cadena vacía, con cero caracteres
String sinDato = null;              // no apunta a ninguna cadena

String copia = materia;             // ambas apuntan a la MISMA cadena
String nueva = new String(materia); // reserva memoria aparte (casi nunca hace falta)
```

Comillas dobles, siempre

'a' con comillas simples es un char, un solo carácter.
"a" con dobles es un String de longitud 1. No son intercambiables.

null no es lo mismo que vacía

"" es una cadena que existe y mide 0. null significa que la variable no apunta a ninguna cadena.

Invocar un método sobre null explota

Si la variable vale null, escribir variable.length() lanza NullPointerException y corta el programa.

```
char letra   = 'J'; // un solo carácter, comillas simples
String texto = "J"; // una cadena de un carácter, comillas dobles

String nombre = null;
System.out.println(nombre.length()); // NullPointerException en ejecución
```

Por qué == no sirve para comparar cadenas

Una variable String guarda una dirección. El operador == compara direcciones, no el texto que hay en ellas. Es exactamente lo que pasaba en C al comparar dos punteros con ==.

```
String a = "Mar del Plata";
String b = "Mar del Plata";
String c = new String("Mar del Plata");
String d = scanner.nextLine();    // el usuario escribe: Mar del Plata

System.out.println(a == b);      // true  -> Java reutilizó la misma
cadena
System.out.println(a == c);      // false -> new reservó otra dirección
System.out.println(a == d);      // false -> la entrada crea otra
dirección

System.out.println(a.equals(b)); // true
System.out.println(a.equals(c)); // true
System.out.println(a.equals(d)); // true <- lo que realmente querías
```

==

Compara DIRECCIONES. Con datos que vienen del teclado o de un archivo, casi siempre da false aunque el texto coincida.

equals()

Compara el CONTENIDO, carácter por carácter. Es el equivalente exacto de strcmp(a, b) == 0.

Truco para evitar el NullPointerException: escribí **"activo".equals(estado)** en lugar de **estado.equals("activo")**. Si estado es null, la primera devuelve false; la segunda corta el programa.

4

Métodos de String

Comparar, medir, buscar, extraer y transformar cadenas

Comparar

Medir

Buscar

Extraer

Comparar: equals, equalsIgnoreCase y compareTo

equals() y equalsIgnoreCase()

Devuelven boolean. equalsIgnoreCase no distingue mayúsculas de minúsculas.

```
String respuesta = "SI";

respuesta.equals("SI");           // true
respuesta.equals("si");           // false
respuesta.equalsIgnoreCase("si"); // true
respuesta.equalsIgnoreCase("Si"); // true
```

compareTo() • el strcmp de Java

Compara alfabéticamente y devuelve un int, no un boolean. Sirve para ordenar.

```
"ana".compareTo("bruno"); // negativo: ana va antes
"bruno".compareTo("ana"); // positivo: bruno va
después
"ana".compareTo("ana");   // 0: son iguales
"Ana".compareTo("ana");   // negativo: 'A' vale menos
que 'a'
```

Valor devuelto por a.compareTo(b)	Significado	Ejemplo
Menor que cero	a va ANTES que b en orden alfabético	"ana".compareTo("bruno")
Igual a cero	a y b son exactamente iguales	"ana".compareTo("ana")
Mayor que cero	a va DESPUÉS que b en orden alfabético	"zoe".compareTo("ana")

Si solo querés saber si dos cadenas son iguales, usá equals(): se lee mejor que compareTo(...) == 0.

length() y concatenación

length()

Devuelve la cantidad de caracteres, contando espacios. Equivale a strlen().

```
String ciudad = "Mar del Plata";

ciudad.length();           // 13
"".length();               // 0
"  ".length();             // 3 (los espacios cuentan)
```

Concatenación con +

Reemplaza a strcat(). Además, une cadenas con números sin convertir nada.

```
String nombre = "Ana";
int edad = 20;

String ficha = nombre + " tiene " + edad + " años";
// "Ana tiene 20 años"
```

La trampa clásica: + con números

Si el operando de la izquierda ya es un String, el + concatena en lugar de sumar. Java evalúa de izquierda a derecha:

```
System.out.println("Total: " + 2 + 3);           // "Total: 23"    <- concatena las dos
System.out.println("Total: " + (2 + 3));         // "Total: 5"      <- los paréntesis mandan
System.out.println(2 + 3 + " puntos");           // "5 puntos"     <- suma y después concatena
```

Transformar: toUpperCase, toLowerCase, trim y replace

Ninguno de estos métodos modifica la cadena original: todos DEVUELVEN una cadena nueva. Si no la asignas a una variable, el resultado se pierde.

toUpperCase()

Copia en MAYÚSCULAS. Reemplaza al bucle con toupper()

"mar del plata" -> "MAR DEL PLATA"

toLowerCase()

Copia en minúsculas

"MAR DEL PLATA" -> "mar del plata"

trim()

Copia sin espacios al inicio ni al final

" Ana " -> "Ana"

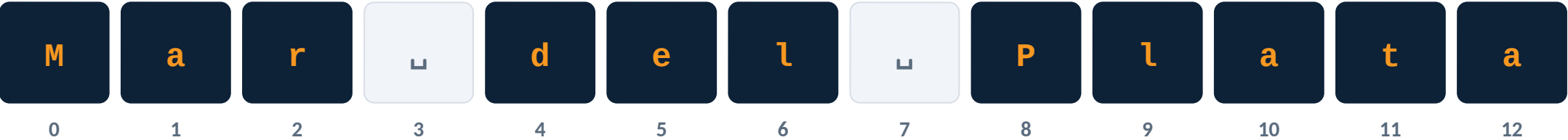
replace(a, b)

Copia reemplazando todas las apariciones

"2026-03-15" -> "2026/03/15"

```
String entrada = " Ana Pérez ";
entrada.trim(); // devuelve la copia... y se pierde
System.out.println(entrada.length()); // sigue valiendo 15
String limpio = entrada.trim(); // ahora sí la guardamos
System.out.println(limpio.length()); // 9
// Encadenar: cada método opera sobre el resultado del anterior
String norm = " Mar Del Plata ".trim().toUpperCase(); // "MAR DEL PLATA"
```


Buscar dentro de una cadena



Los índices empiezan en 0, igual que en los arreglos de C.

Método	Qué devuelve	Sobre "Mar del Plata"
<code>indexOf(String s)</code>	Posición de la PRIMERA aparición, buscando desde el principio	<code>indexOf("a")</code> -> 1
<code>lastIndexOf(String s)</code>	Posición de la ÚLTIMA aparición, buscando desde el final	<code>lastIndexOf("a")</code> -> 12
<code>indexOf(String s, int desde)</code>	Primera aparición a partir de una posición dada	<code>indexOf("a", 2)</code> -> 10
<code>contains(String s)</code>	true o false según si la subcadena aparece	<code>contains("del")</code> -> true
<code>charAt(int pos)</code>	El carácter en esa posición. Reemplaza a <code>cadena[i]</code>	<code>charAt(8)</code> -> 'P'

Si no encuentra nada, `indexOf()` y `lastIndexOf()` devuelven -1. Nunca lanzan error: siempre hay que preguntar por el -1.

Extraer: substring()

Devuelve una porción de la cadena. Existe en dos versiones, y la diferencia entre incluir o no el último índice es la fuente habitual de errores.

substring(int desde)

Desde esa posición hasta el final de la cadena.

```
"Mar del Plata".substring(8) -> "Plata"
```

substring(int desde, int hasta)

Desde «desde» inclusive, hasta «hasta» EXCLUSIVE.

```
"Mar del Plata".substring(4, 7) -> "del"
```



substring(4, 7): entra el 4, entran el 5 y el 6, NO entra el 7.

```
String legajo = "UTN-2026-1834";  
String universidad = legajo.substring(0, 3); // "UTN"  
String anio        = legajo.substring(4, 8); // "2026"  
String numero       = legajo.substring(9);   // "1834"  
legajo.substring(4, 8).length();             // 4 caracteres: hasta - desde
```

Otros métodos que vas a usar seguido

Método	Devuelve	Qué hace	Ejemplo
<code>startsWith(String p)</code>	<code>boolean</code>	Si la cadena comienza con ese prefijo	<code>"UTN-2026".startsWith("UTN") -> true</code>
<code>endsWith(String s)</code>	<code>boolean</code>	Si la cadena termina con ese sufijo	<code>"foto.png".endsWith(".png") -> true</code>
<code>isEmpty()</code>	<code>boolean</code>	Si la longitud es exactamente cero	<code>"".isEmpty() -> true</code>
<code>isBlank()</code>	<code>boolean</code>	Si está vacía o solo tiene espacios	<code>" ".isBlank() -> true</code>
<code>repeat(int n)</code>	<code>String</code>	Repite la cadena n veces	<code>"-".repeat(5) -> "-----"</code>
<code>concat(String s)</code>	<code>String</code>	Equivale al operador + entre cadenas	<code>"Mar".concat(" del Plata")</code>

Diferencia sutil: `" .isEmpty()` da false (tiene 3 caracteres), pero `" .isBlank()` da true. Para validar lo que escribe el usuario querés `isBlank()`.

5

Cadenas modificables

Por qué un String no se puede cambiar, y qué usar cuando hace falta

El problema

StringBuilder

Métodos

Cuándo usar cada uno

Un String no se puede modificar

En C podías escribir `cadena[0] = 'X'` y cambiar la cadena en el lugar. En Java eso no existe: una vez creada, una cadena no cambia nunca. Todos los métodos DEVUELVEN una cadena nueva.

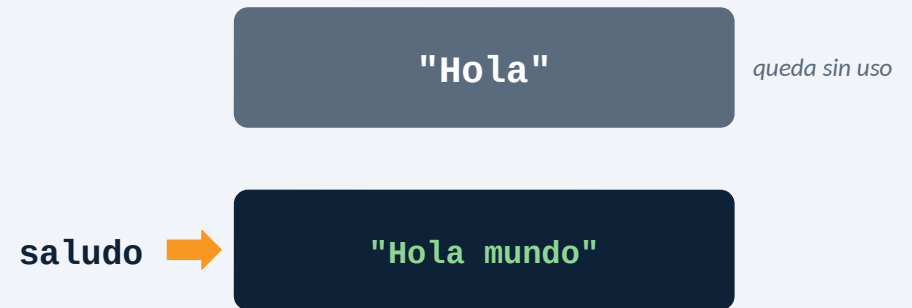
```
// En C esto funcionaba
char saludo[20] = "Hola";
saludo[0] = 'M';           // ahora dice "Mola"

// En Java no hay forma de hacerlo
String saludo = "Hola";
saludo.charAt(0) = 'M';    // ERROR: no compila

saludo.concat(" mundo");
System.out.println(saludo);           // "Hola": no cambió

saludo = saludo.concat(" mundo");
System.out.println(saludo);           // "Hola mundo"
```

QUÉ PASA EN MEMORIA



La variable pasa a apuntar a otra dirección. La cadena vieja queda sin nadie que la use, y Java libera esa memoria sola: no hay `free()`.

Seguridad

Una cadena que ya validaste no puede ser alterada después por otro método.

Sin desbordes

El clásico buffer overflow de C, escribir más allá del `char[100]`, no puede ocurrir.

No hay que reservar

Nunca calculás cuántos caracteres entran ni pedís memoria por adelantado.

El costo

Cada modificación crea una cadena nueva. En un bucle largo, eso se paga caro.

StringBuilder: la cadena que sí se puede modificar

Es un buffer de caracteres que se modifica en el lugar, sin generar cadenas nuevas. Es lo más parecido al `char[]` de C, pero crece solo cuando hace falta.

No se puede

```
StringBuilder sb = "abc";
```

Se crea con new

```
StringBuilder sb = new StringBuilder("abc");
```

```
StringBuilder sb = new StringBuilder("Mar");

sb.append(" del Plata");           // sb = "Mar del Plata"
sb.insert(0, ">> ");                // sb = ">> Mar del Plata"
sb.reverse();                      // sb = "atlaP led raM >>"
sb.reverse();                      // vuelve a ">> Mar del Plata"

// Siempre es el MISMO buffer: no se reasigna nada
String resultado = sb.toString();  // pasarlo a String al final
```

Se modifica en el lugar

Los métodos cambian el buffer, sin crear una cadena nueva cada vez.

new reserva la memoria

Es lo más cercano a un malloc, pero no hay que calcular el tamaño ni liberarlo.

No se compara con equals()

Para comparar contenido hay que convertirlo con toString() primero.

toString() lo cierra

Cuando terminaste de armar la cadena, la pasás a String.

Métodos de StringBuilder

Método	Qué hace	Ejemplo partiendo de "Java"
<code>append(x)</code>	Agrega al final. Acepta cualquier tipo de dato	<code>append(" 21") -> "Java 21"</code>
<code>insert(int pos, x)</code>	Inserta en la posición indicada, corriendo el resto	<code>insert(0, "i") -> "iJava"</code>
<code>delete(int ini, int fin)</code>	Borra el rango [ini, fin), igual que substring	<code>delete(0, 2) -> "va"</code>
<code>deleteCharAt(int pos)</code>	Borra un solo carácter	<code>deleteCharAt(0) -> "ava"</code>
<code>replace(int i, int f, String s)</code>	Reemplaza el rango por la cadena indicada	<code>replace(0, 4, "Kotlin") -> "Kotlin"</code>
<code>setCharAt(int pos, char c)</code>	Cambia un carácter. Es el <code>cadena[i] = 'X'</code> de C	<code>setCharAt(0, 'K') -> "Kava"</code>
<code>reverse()</code>	Invierte el orden de todos los caracteres	<code>reverse() -> "avaJ"</code>
<code>length()</code>	Cantidad de caracteres del buffer	<code>length() -> 4</code>
<code>toString()</code>	Devuelve el contenido como un String	<code>toString() -> "Java"</code>

Los métodos devuelven el propio buffer, así que se pueden encadenar: `sb.append("Mar").append(" del ").append("Plata");`



Arrays

Arreglos de tamaño fijo: declarar, reservar, inicializar y recorrer

Qué son

Declarar

Reservar

Recorrer

¿Qué es un array?

Un arreglo contiene varios elementos ordenados del mismo tipo, accesibles por su posición. Es el mismo concepto que en C, con dos diferencias importantes.



Un solo tipo

Todos los elementos son del mismo tipo: `int[]`, `String[]`, `double[]`.

Tamaño fijo

Se define al reservarlo y no cambia. Igual que en C.

Sabe cuánto mide

Novedad respecto de C: el arreglo lleva su propia longitud en `.length`.

Es un tipo de referencia

La variable guarda una dirección, como un `int *` en C.

En C tenías que pasar el tamaño aparte: `procesar(notas, 5)`. En Java no hace falta: el arreglo lo lleva adentro.

Declarar, reservar e inicializar

1 DECLARAR

Se indica el tipo que va a contener, seguido de corchetes, y el nombre de la variable.

```
int[] notas;  
String[] nombres;  
  
// También es válido, y es la  
// forma que usabas en C:  
int notas[];
```

2 RESERVAR

Con new se pide la memoria, indicando el tipo y la cantidad de elementos.

```
notas = new int[5];  
  
// Es el malloc de C, pero  
// sin sizeof y sin free.  
// Los 5 arrancan en 0.
```

3 INICIALIZAR

Se cargan los valores, uno por uno o todos juntos con llaves.

```
notas[0] = 8;  
notas[1] = 6;  
  
// o todo junto, sin new:  
int[] notas = {8, 6, 10, 4, 9};
```

Diferencia con C: allá escribías `int notas[5]`; y el tamaño iba en la declaración. En Java el tamaño va en el `new`.

Las formas abreviadas y los valores por defecto

Forma	Código	Resultado
Declarar y reservar	<code>int[] notas = new int[5];</code>	5 elementos, todos en 0
Declarar, reservar e inicializar	<code>int[] notas = {8, 6, 10, 4, 9};</code>	5 elementos con esos valores; el tamaño se deduce
Con new y valores explícitos	<code>int[] n = new int[]{8, 6, 10};</code>	Necesario al pasar un arreglo directo a un método
Arreglo de cadenas	<code>String[] nombres = new String[3];</code>	3 posiciones, todas en null

Valores por defecto según el tipo

int, long, short, byte

0

double, float

0.0

char

'\u0000'

boolean

false

String

null

Otra diferencia con C: allá un arreglo sin inicializar contenía basura. Java garantiza que arranca en el valor por defecto.

length: sin paréntesis en arreglos, con paréntesis en String

Atención

Es la confusión más frecuente de esta clase, y produce un error de compilación difícil de leer.

Arreglo

notas.length

Es un DATO guardado adentro del arreglo. Sin paréntesis.

String

texto.length()

Es una FUNCIÓN que hay que invocar. Con paréntesis.

```
int[] notas = {8, 6, 10, 4, 9};
String ciudad = "Mar del Plata";

System.out.println(notas.length);      // 5    correcto
System.out.println(ciudad.length());   // 13   correcto

System.out.println(notas.length());    // ERROR: cannot find symbol
System.out.println(ciudad.length);     // ERROR: cannot find symbol
```

Cómo recordarlo

El arreglo GUARDA su tamaño: es un dato, y a los datos se accede sin paréntesis.

El String lo CALCULA: es una función, y las funciones se invocan con paréntesis, aunque no reciban nada.

Acceder a las posiciones y asignar valores

Se accede a cada elemento por su índice, entre corchetes, exactamente igual que en C. Una vez en la posición, se lee o se asigna como cualquier otra variable.

```
String[] materias = new String[4];

materias[0] = "Programación II";
materias[1] = "Análisis Matemático II";
materias[2] = "Física I";
materias[3] = "Inglés I";

System.out.println(materias[0]);           // Programación
II
System.out.println(materias[materias.length - 1]); // Inglés I

materias[1] = "Álgebra";           // sobrescribe el valor anterior
```

Primer elemento

array[0]

Nunca array[1]. El índice arranca en cero.

Último elemento

array[array.length - 1]

Nunca array[array.length]: esa posición no existe.

Acá Java se porta mejor que C

En C, salirse del arreglo leía basura o corrompía la memoria sin avisar. Java corta el programa con un mensaje claro: `ArrayIndexOutOfBoundsException`. Sigue siendo un error, pero al menos te enterás en el acto.

Recorrer un arreglo: while y for

Igual que en C

El patrón es el mismo de siempre: un índice que arranca en 0 y avanza mientras sea menor que la longitud.

Con while

```
int[] notas = {8, 6, 10, 4, 9};

int i = 0;
while (i < notas.length) {
    System.out.println("Nota " + i + ": " + notas[i]);
    i++;
}
```

Con for

```
int[] notas = {8, 6, 10, 4, 9};

for (int i = 0; i < notas.length; i++) {
    System.out.println("Nota " + i + ": " + notas[i]);
}
```

Siempre < nunca <=

Con length = 5, los índices válidos son 0, 1, 2, 3 y 4. Escribir `i <= notas.length` llega hasta el 5 y corta el programa en la última vuelta.

Ya no hace falta pasar el tamaño

En C escribías `for (i = 0; i < cantidad; i++)` porque el arreglo no sabía cuánto medía. En Java el propio arreglo te da la longitud con `.length`, y no puede quedar desactualizada.

Recorrer con for-each

Novedad en Java

Recorre todos los elementos sin manejar el índice a mano. Se lee: «para cada nota dentro de notas».

```
for ( tipo variable : arreglo ) { ... }
```

```
int[] notas = {8, 6, 10, 4, 9};

// Recorrido tradicional
for (int i = 0; i < notas.length; i++) {
    System.out.println(notas[i]);
}

// Mismo resultado, con for-each
for (int nota : notas) {
    System.out.println(nota);
}

// Funciona con cualquier tipo
String[] materias = {"Programación II", "Física I"};
for (String materia : materias) {
    System.out.println(materia.toUpperCase());
}
```

Usalo cuando

Solo necesitas LEER cada elemento, de principio a fin, y no te importa en qué posición está.

No sirve cuando

- Necesitás saber el índice.
- Querés modificar el arreglo: asignar a la variable del for-each no cambia el original.
- Querés recorrerlo al revés o salteado.

Utilidades listas para usar: java.util.Arrays

Es una biblioteca de funciones para arreglos, equivalente a lo que `string.h` era para cadenas. Se invocan escribiendo `Arrays` adelante, y hay que importarla: `import java.util.Arrays;`

Función	Qué hace	Ejemplo sobre {8, 6, 10}
<code>Arrays.toString(a)</code>	Devuelve el contenido como texto legible	<code>"[8, 6, 10]"</code>
<code>Arrays.sort(a)</code>	Ordena el arreglo de menor a mayor, en el mismo arreglo	<code>{6, 8, 10}</code>
<code>Arrays.fill(a, v)</code>	Llena todas las posiciones con el mismo valor	<code>fill(a, 0) -> {0, 0, 0}</code>
<code>Arrays.copyOf(a, n)</code>	Devuelve una copia nueva del tamaño indicado	<code>copyOf(a, 2) -> {8, 6}</code>
<code>Arrays.equals(a, b)</code>	Compara contenido elemento por elemento	<code>true</code> si coinciden todos

```
import java.util.Arrays;
int[] notas = {8, 6, 10, 4, 9};
System.out.println(notas);           // [I@1b6d3586 <- la dirección
System.out.println(Arrays.toString(notas)); // [8, 6, 10, 4, 9]
Arrays.sort(notas);
System.out.println(Arrays.toString(notas)); // [4, 6, 8, 9, 10]
```


Cierre de la Clase N° 2

Lo que tenés que llevarte de hoy

La función de C es un método

Mismo concepto, dentro del bloque class y con la palabra static.

Los parámetros van por valor

No hay punteros: para devolver un resultado, usá return.

Nunca compares cadenas con ==

== compara direcciones. equals() es el strcmp(a,b)==0 de Java.

Las cadenas no se modifican

Todo método devuelve una copia. Para cambiarlas, StringBuilder.

length vs length()

Sin paréntesis en arreglos; con paréntesis en String.

El índice arranca en 0

El último elemento es [length - 1], igual que en C.

Próxima clase: qué significa realmente la palabra class, y qué es esa otra forma de escribir métodos sin static.

Documentación oficial: docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/String.html